Linux Privilegien

Grundlagenpraktikum zur IT-Sicherheit (WiSe 2023/2024)

Lehrstuhl für Netz- und Datensicherheit
Ruhr-Universität Bochum

Nico S**** Sabri Öztürk

1. Mai 2026

Zusammenfassung

Dieser Versuch beschäftigt sich mit den Grundlagen von Linux-Privilegien. Linux ist ein weit verbreitetes Betriebssystem, das eine robuste Sicherheitsinfrastruktur bietet, um den Zugriff auf Ressourcen und Funktionen zu regeln. Doch wie funktionieren Linux-Privilegien? Wie werden Benutzer und Prozesse autorisiert, auf bestimmte Dateien, Verzeichnisse und Systemressourcen zuzugreifen? Welche Berechtigungen können von Angreifern ausgenutzt werden? Diese Fragen und weitere Fragen werden in diesem Protokoll behandelt.

Inhaltsverzeichnis

1 Einleitung **1**

1.1 Glossar 1

2 Aufbau **2**

3 Durchführung **3**

3.1 Erkundung der Umgebung 3

3.2 su und sudo 5

3.3 Setzen einfacher Berechtigungen 7

3.4 Gruppenprivilegien 9

3.5 Fallbeispiel cron 11

3.6 Fallbeispiel shadow 12

4 Zusammenfassung und Fazit **13**

5 LLM **14**

1 Einleitung

Ein Computer ist ein nützliches Gerät. Es können auf ihm Texte verfasst, Programme geschrieben und auch ausgeführt werden. Doch solch eine Vielseitigkeit birgt Risiken. Auf einem Computer können meist mehrere Nutzer angelegt werden. Aber nicht jeder dieser Nutzer soll auf alle Dateien auf dem Computer Zugriff haben. So soll ein Kind beispielsweise nicht die Steuererklärung der Eltern bearbeiten können.

Um dies zu ermöglichen gibt es Nutzerrechte, Gruppen und Gruppenrechte. Über diese Rechte lassen sich Dateien bestimmten Nutzern zuordnen. Es ist also möglich, eine Datei für Nutzer A freizugeben und für Nutzer B zu sperren. Es können auch Nutzergruppen erstellt werden, wodurch alle Nutzer einer Gruppe in der Lage sind, die gewählten Dateien zu verändern. Jedoch können Nutzer die nicht der geforderten Gruppe angehöhren oft nur die Datei einsehen oder haben gar keinen Zugriff.

Was ein Nutzer darf und was nicht wird mit Privilegien bestimmt. Doch auch Privilegien müssen durchdacht sein. Wenn einem Nutzer die falschen Privilegien gegeben werden, kann dieser womöglich nicht einmal mehr in seinem eigenen Home-Verzeichnis Dateien anlegen. Oder aber der Nutzer ist in der Lage, auch Daten von anderen Nutzer zu lesen, auf die er keinen Zugriff haben sollte.

Zudem wird ein weiteres nützliches Feature erkundet. Der Cronjob kann Befehle und Programme periodisch ausführen. Es ist möglich in einer Datei festzulegen, was in einem bestimmten Monat an einem bestimmt Tag zu einer ausgewählten Uhrzeit passieren soll. So kann beispielsweise regelmäßig ein Backup erstellen werden. Doch auch hier muss auf die richtigen Privilegien geachtet werden, da ein Angreifer ein periodisches Programm zu seinen Zwecken umbauen könnte.

Es ist also wichtig, Nutzerprivilegien in allen Einzelheiten zu verstehen. Dies soll mit der Hilfe dieses Versuches geschehen. Wir werden einen Einblick in das Setzten und Entfernen von Privilegien erhalten, wie Nutzergruppen erstellt und bearbeitet werden und was passiert, wenn falsche Privilegien vergeben wurden.

1.1 Glossar

Flag ist ein weg das Verhalten eines Befehls in der Shell anzupassen

Kernel ist der Kern eines Betriebsystems

uname ist ein Befehl, zum Gewinnen von Systeminformationen

2 Aufbau

Die folgenden Versuche wurden im Netzlabor ID 2/168 unternommen.
Wir führen den Versuch zu zweit als Gruppe 43 am PC mit der IP-Adresse 192.168.1.42 durch.
Das Betriebsystem ist ein Ubuntu 22.04 LTS.

Es wurden verschiedene Programme und Befehle genutz. Diese sind:

Software	**Version**
uname	GNU coreutils 8.32
grep	GNU grep 3.7
ls	GNU coreutils 8.32
echo	GNU coreutils 8.32
id	GNU coreutils 8.32
touch	GNU coreutils 8.32
su	2.37.2
Sudo-Version	1.9.9
history	GNU History 8.1
chmod	GNU coreutils 8.32
rm	GNU coreutils 8.32
cat	GNU coreutils 8.32
chown	GNU coreutils 8.32

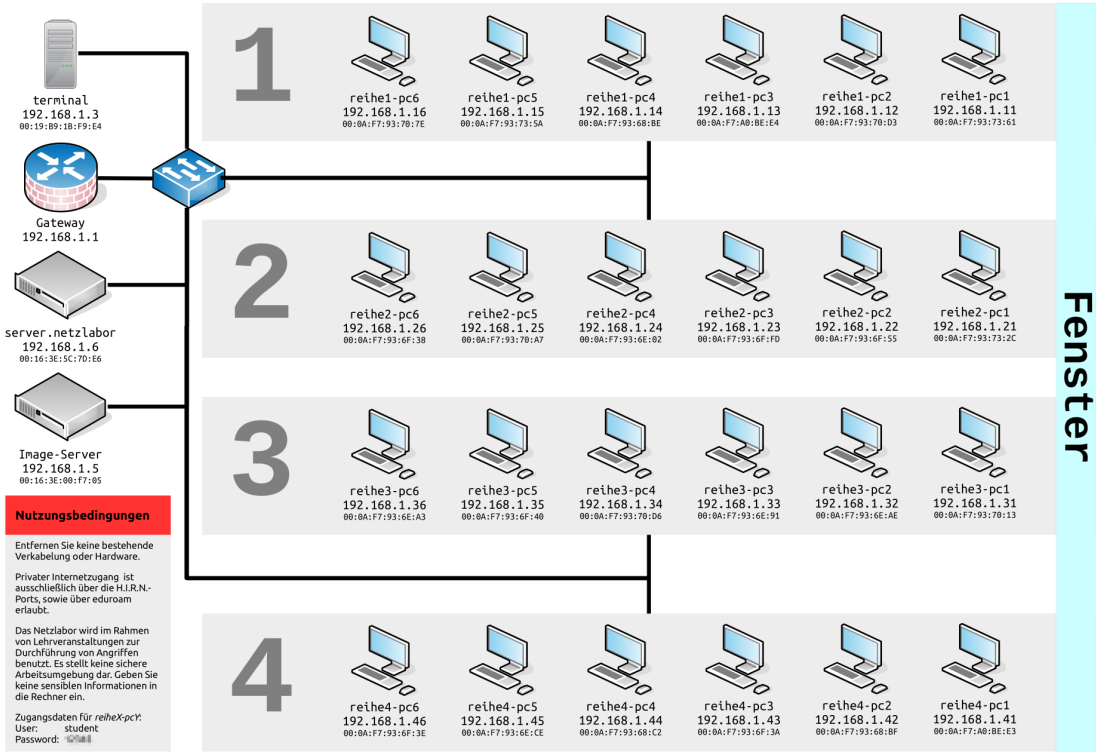

Abbildung 1: Netzplan Raum ID 2/168, Stand 01.05.2017

Die Aufgabenstellung [2] wurde bereitgestellt über die Veranstaltungsseite [1]

3 Durchführung

Im Folgenden werden die einzelnen Aufgaben durchgeführt, erläutert und das Ergebnis präsentiert.

3.1 Erkundung der Umgebung

In einer unbekannten Softwareumgebung ist das Erkunden des Betriebsystems und der Dateisysteme essenziell. So können mitunter wichtige Informationen zu Berechtigungen und vorhandenen Befehlen gewonnen werden.

Mit dem Befehl *uname* können Systeminformationen asugegeben werden. So kann mit Hilfe der Flag *-s* der Name des Kernels asugegeben werden. Über die Flag *-n* wird der Netzwerkname der Maschine ausgegeben.

```
nikki@Reihe4-PC2:~/Schreibtisch$ uname -sn
Linux Reihe4-PC2
```

Dann wird die auf den Nutzer nikki beziehende Zeile aus */etc/passwd* ausgegeben. In der Datei */etc/shadow* stehen Nutzernamen mit den jeweiligen Passwoörten in gehasheder Form. Schließlich werden die hinterlegten Standard-Konsolen der Nutzer ausgegeben.

```
nikki@Reihe4-PC2:~/Schreibtisch$ grep nikki /etc/passwd
nikki:x:1001:1001:Nikki Dangler, IA 0/95,,:/home/nikki:/bin/bash
nikki@Reihe4-PC2:~/Schreibtisch$ cat /etc/shadow
root:$6$E9B.
    uuqHXtpDTpwx$pRl44lSLZAKKoVIhoPVbESwoqrNkMo9sFNHFCqCJoePcjWkTRvd66BdGVi9us4Zogrj5XM
3sIiJsJVo3ICq131:19654:0:99999:7:::
[...]
student:$y$j9T$UyRlqBji3D8bIZlGZcYyA1$uIACKneK8Zrpo1yx9tPw4VJ9xfHe0xoxA1dp.SwKBi5
    :19423:0:99999:7:::
nikki:$6$F.Wiz.U8VAhWbfs0$dOVdICNmoPL7LI21eg//dfFI/
    RWu1xA9rQuH9gBbxgHQxsBXeerKTtuyQFWsepAnKB37hEcvT5JjPuL2kFF.41:19654:0:99999:7:::
alice:*:19654:0:99999:7:::
bob:*:19654:0:99999:7:::
jonas:*:19654:0:99999:7:::
nikki@Reihe4-PC2:~$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
[...]
student:x:1000:1000:,,,:/home/student:/bin/bash
nikki:x:1001:1001:Nikki Dangler, IA 0/95,,:/home/nikki:/bin/bash
alice:x:1002:1003:Alice, ID 6/210,,:/home/alice:/bin/bash
bob:x:1003:1004:Bob, ID 6/211,,:/home/bob:/bin/sh
jonas:x:1004:1005:Jonas,,,:/home/jonas:/usr/bin/b4sh
```

Der absolute Pfad zum Home-Verzeichnis von nikki wird angegeben und die Berechtigungen darauf werden gelistet. Für das Home-verzeichnis von Alice sind die Berechtigungen *drwxr-xrwx* gesetzt. Das *d* bedeutet, dass es sich um ein Verzeichnies (englisch 'directory') handelt. Die folgenden drei Buchstaben *rwx* sagen aus, dass der Eigentümer des Verzeichnis' Lese-, Schreib- und Ausführberechtigungen besitzt. Der Besitzer kann also sehen, welche Dateien und Verzeichnisse in diesem Verzeichnis sind, darf neue Dateien anlegen und kann in das Verzeichnis wechseln. Die gleichen Berechtigungen sind in den letzten drei Buchstaben wiederzufinden. Dies bedeutet, dass jeder andere Nutzer ebenso diese Berechtigungen hat. Es kann also jeder Benutzter die gleichen Aktionen ausführen wie Alice selber. Die Berechtigung *r-x* sagt, dass Nutzer der Gruppe *Alice* die vorhandenen Dateien anzeigen darf und in das Verzeichnis wechseln darf. Es können jedoch keine neuen Dateien erstellt werden, da die Berechtigung *w* fehlt.

```
nikki@Reihe4-PC2:~/Schreibtisch$ cd ~
nikki@Reihe4-PC2:~$ pwd
/home/nikki
nikki@Reihe4-PC2:~$ ls -l /home/
insgesamt 20
drwxr-xrwx  2 alice   alice   4096 Okt 24 12:07 alice
drwxr-x---  2 bob     bob     4096 Okt 24 12:07 bob
drwxr-x---  2 jonas   jonas   4096 Okt 24 12:07 jonas
drwxr-x--- 15 nikki   nikki   4096 Okt 25 09:05 nikki
drwxr-x--- 16 student student 4096 Okt 12 19:55 student
nikki@Reihe4-PC2:/home$
```

Über den Befehl *alias* kann einer Zeichenkette ein Befehl zugeordnet werden. So lassen sich komplexe oder lange Befehle kürzen und einfacher nutzen. Die Datei *.bashrc* enthält eine Reihe von Befehlen, welche als Konfiguration für die Bash-Shell dienen. Es kann so ein dauerhafter *alias* eingefügt werden, der in jeder neu geöffneten Konsole vorhanden ist. Der Befehl 'source' gewährleistet, dass diese Änderungen in der aktuellen Konsole sofort wirksam sind.

```
nikki@Reihe4-PC2:~$ echo alias ll=\"ls -l\" >> /home/nikki/.bashrc && source /home/nikki
    /.bashrc
```

Der absolute Pfad zum Home-Verzeichnis von root ist */root/*. Wenn Versucht wird, als standard Benutzer, in das Home-Verzeichnis von root zu schauen gibt es eine Fehlermeldung, denn standard Nutzer haben nicht die nötigen Berechtigungen um in das Verzeichnis zu schauen. Dies ist daran zu erkennen, dass über den etablierten alias, der Inhalt des Verzeichnis' nicht angezeigt wird. Zudem können die Berechtigungen angezeigt werden. Die Berechtigungen *drwx*—— erlauben es alleine dem Besitzer des Verzeichnis' die enthaltenen Dateien zu sehen, neue Dateien anzulegen und in das Verzeichnis zu wechseln.

```
nikki@Reihe4-PC2:~$ ll /root/
ls: Öffnen von Verzeichnis '/root/' nicht möglich: Keine Berechtigung
nikki@Reihe4-PC2:~$ ll /
[Gekürtzt um momentan unwichtige Verzeichnisse]
drwx------  1 root root    4096 Nov 24  2022 root
[Gekürtzt um momentan unwichtige Verzeichnisse]
nikki@Reihe4-PC2:~$ ll /home/
insgesamt 20
drwxr-xrwx  2 alice   alice   4096 Okt 24 12:07 alice
drwxr-x---  2 bob     bob     4096 Okt 24 12:07 bob
drwxr-x---  2 jonas   jonas   4096 Okt 24 12:07 jonas
drwxr-x--- 15 nikki   nikki   4096 Okt 25 09:30 nikki
drwxr-x--- 16 student student 4096 Okt 12 19:55 student
nikki@Reihe4-PC2:~$
```

Wir sind als nikki angemeldet. Seine User ID ist '1001'. Seine Gruppen ID ist '1001'. Das ist seine primäre Gruppe. Der Nutzer nikki ist zusätzlich ein Mitglied der Gruppe 'it_services' mit der Gruppen ID '1002'.

```
nikki@Reihe4-PC2:~$ id
uid=1001(nikki) gid=1001(nikki) Gruppen=1001(nikki),1002(it_services)
```

3.2 su und sudo

Mit den Befehlen *su* und *sudo* können die Root-Berechtigungen erlangt werden.

Der Befehl *touch /home/bob/test* wird ausgeführt. Die Fehlermeldung besagt, dass die Berechtigung für die Ausführung des Befehls nicht vorhanden sei.

```
nikki@Reihe4-PC2:~$ touch /home/bob/test
touch: '/home/bob/test' kann nicht berührt werden: Keine Berechtigung
```

Benutzer 'nikki' ist nicht in der Liste der Benutzer, die sudo-Berechtigungen haben, und daher kann er keine root-Privilegien über sudo erhalten.

```
nikki@Reihe4-PC2:~$ sudo touch /home/bob/test
[sudo] Passwort für nikki:
nikki ist nicht in der sudoers-Datei. Dieser Vorfall wird gemeldet.
```

Es wird eine Konsole im Kontext des Nutzers root gestartet. Das Passwort von root wird gefragt.

```
nikki@Reihe4-PC2:~$ su -
Passwort:
```

Der Befehl *id* gibt 'root' als Nutzername aus. Das stimmt mit dem Nutzer, der am Anfang jeder Eingabezeile angezeigt wird, überein. Im root Verzeichnis befinden wir uns gerade.

```
root@Reihe4-PC2:~# id
uid=0(root) gid=0(root) Gruppen=0(root)
root@Reihe4-PC2:~# pwd
/root
```

Der Befehl *usermod* wird genutzt, um Anpassungen an den Eigenschaften eines Benutzerkontos vorzunehmen. Mit der Verwendung von *-a* bleibt der Benutzer gleichzeitig Mitglied aller anderen Gruppen, denen er bereits angehört. Die Option *-G* ordnet in diesem Kontext der Gruppe 'sudo' zu, zu der letztendlich 'nikki' hinzugefügt wird.

```
root@Reihe4-PC2:~# usermod -a -G sudo nikki
```

Die 20 zuletzt ausgeführten Befehle werden angezeigt. Die ersten und zweiten ausgeführten 'touch-Befehle' werden nicht angezeigt, da sie nicht als root Nutzer ausgeführt worden sind und damit in einer anderen Umgebung.

```
root@Reihe4-PC2:~# history | tail -n 20
    1  passwd student
    2  su student
    3  exit
    4  id
    5  pwd
    6  usermod -a -G sudo nikki
    7  history | tail -n 20
```

Mit *exit* wird zur unprivilegierten Konsole zurückgekehrt.

```
root@Reihe4-PC2:~# exit
```

Damit kein Systemneustart benötigt wird, wird der Befehl *exec su -l nikki* ausgeführt, der bei der Änderung einer Konsole wieder ausgeführt werden muss. Schließlich wird überprüft, ob 'nikki' in der neuen Gruppe ist. Group-ID von sudo ist 27.

```
nikki@Reihe4-PC2:~$ exec su -l nikki
Passwort:
nikki@Reihe4-PC2:~$ id
uid=1001(nikki) gid=1001(nikki) Gruppen=1001(nikki),27(sudo),1002(it_services)
```

Der Befehl *touch /home/bob/test* funktioniert nicht, da der Nutzer 'nikki' nicht die nötige Berechtigung hat. Mit dem Befehl *sudo touch /home/bob/test* funktioniert es, da der Befehl im Kontext des root-Benutzers ausgeführt wird.

```
nikki@Reihe4-PC2:~$ touch /home/bob/test
touch: '/home/bob/test' kann nicht berührt werden: Keine Berechtigung
nikki@Reihe4-PC2:~$ sudo touch /home/bob/test
[sudo] Passwort für nikki:
```

Die 20 zuletzt ausgeführten Befehle werden angezeigt. Der ausgeführte 'usermod-Befehl' wird nicht angezeigt, da sie nicht in 'nikki' Nutzer ausgeführt worden ist und damit in einer anderen Umgebung. Die 'sudo-Befehle' sind von dem Nutzer 'nikki' ausgeführt worden, darum sind sie zu sehen.

```
nikki@Reihe4-PC2:~$ history | tail -n 20
   49  id
   50  clear
   51  su -
   52  exec su -l nikki
   53  sudo tocch hi
   54  sudo touch hi
   55  id
   56  rm hi
   57  touch /home/bob/test
   58  sudo touch /home/bob/test
   59  clear
   60  exit
   61  touch /home/bob/test
   62  sudo touch /home/bob/test
   63  su -
   64  exec su -l nikki
   65  id
   66  touch /home/bob/test
   67  sudo touch /home/bob/test
   68  history | tail -n 20
```

Ob Bob auf sein eigenes Home-Verzeichnis zugreifen kann, wird getestet. Dazu wäre der Befehl *su -l bob* nützlich, jedoch wird das Passwort von Bob verlangt, das wir nicht kennen. Um das zu vermieden fügen wir *sudo* am Anfang des Befehls hinzu. Damit würde das Passwort für den Benutzer, der sudo ausführt, benötigt. In unserem Fall ist das Passwort schon gecached. Die für bob geöffnete Konsole sieht anders aus, als die von uns verwendete /bin/bash, da bob eine andere Standardshell benutzt.

```
nikki@Reihe4-PC2:~$ su -l bob
Passwort:

su: Legitimierungsfehler
nikki@Reihe4-PC2:~$
nikki@Reihe4-PC2:~$ sudo su -l bob
```

Mit dem Befehl *id* wird kontrolliert, ob wir tatsäclich als 'bob' angemeldet sind. Über den Befehl *ls -la /home/* kann eine Liste mit Berechtigungen der Dateien und Verzeichnisse im Verzeichnis */home/* ausgegeben werden. Wie zu sehen ist, sollte der Nutzer 'bob' Zugriff auf sein Home-Verzeichnis haben. Dies kann überprüft werden, indem mit *touch* eine Datei */home/bob/test* versucht wird zu erstellen. Jedoch kann, durch einen nicht bekannten Fehler, 'bob' nicht auf sein Home-Verzeichnis zugreifen. Auf das Verzeichnis */home/jonas* sollte 'bob' keinen Zugriff haben und besitzt diesen auch nicht. Zuletzt wird Kontext verlassen und mit *id* überprüft, ob wir wieder als 'nikki' angemeldet sind.

```
$ id
uid=1003(bob) gid=1004(bob) Gruppen=1004(bob)
$ ls -la /home/
drwxr-x---  2 bob     bob     4096 Okt 24 12:07 bob
drwxr-x---  2 jonas   jonas   4096 Okt 24 12:07 jonas
[nicht relevante Verzeichnisse sind ausgeblendet]
$ touch /home/bob/test
touch: '/home/bob/test' kann nicht berührt werden: Keine Berechtigung
$ touch /home/jonas/test
touch: '/home/jonas/test' kann nicht berührt werden: Keine Berechtigung
$ exit
nikki@Reihe4-PC2:~$ id
uid=1001(nikki) gid=1001(nikki) Gruppen=1001(nikki),27(sudo),1002(it_services)
```

3.3 Setzen einfacher Berechtigungen

Mithilfe der Befehle *chown* und *chmod* werden Privilegien angepasst.

Aktuell sind die Berechtingungen für Alices *drwxr-xrwx*. Dies bedeutet, dass nicht nur der Inhaber des Verzeichnis' Dateien erstellen, ändern und löschen darf, sonder jeder Nutzer.

```
nikki@Reihe4-PC2:~$ ll /home/
insgesamt 20
drwxr-xrwx  2 alice   alice   4096 Okt 24 12:07 alice
drwxr-x---  2 bob     bob     4096 Okt 24 12:07 bob
drwxr-x---  2 jonas   jonas   4096 Okt 24 12:07 jonas
drwxr-x--- 15 nikki   nikki   4096 Okt 25 09:30 nikki
drwxr-x--- 16 student student 4096 Okt 12 19:55 student
```

Die Berechtigungen auf Alices Home-Verzeichnis sollten die gleiche, wie bei den anderen Verzeichnissen sein. Die dezimale Darstellung der Berechtigung *drwxr-x—* ist 750. Die 7 steht für *rwx*, die 5 für *r-x* und die 0 für —. Sie erlaubt nur dem Inhaber 'alice' vollen Zugriff und der Nutzergruppe 'alice' *r-x*. Allen anderen Nutzern wird der Zugriff verweigert.

```
nikki@Reihe4-PC2:~$ sudo chmod 750 /home/alice
nikki@Reihe4-PC2:~$ cd /home
nikki@Reihe4-PC2:/home$ ll
insgesamt 20
drwxr-x---  2 alice   alice   4096 Okt 24 12:07 alice
nikki@Reihe4-PC2:/home$
```

Das erste Zeichen 'd' zeigt an, dass es sich um ein Verzeichnis handelt. Die nächsten drei Zeichen 'rwx' stehen für die Berechtigungen des roots des Verzeichnisses. Die restlichen '-' Zeichen stehen dafür, dass alle andere keine Berechtigung auf dem Verzeichnis haben. Also das Verzeichnis '/root/' ist nur für den Root-Benutzer lesbar, schreibbar und ausführbar. Es hat keine Berechtigungen für andere Benutzer, aus Gründen der Sicherheit.

Die Fehlermeldung

```
/usr/local/bin/backItUp . sh : Permission denied on execution
```

deutet darauf hin, dass die Datei */usr/local/bin/backItUp.sh* nicht ausgeführt werden kann, da auf der Datei nicht die entsprechenden Berechtigungen gestzt sind. Durch *ll /usr/bin/backItUp.sh* wird klar, dass die Ausführberechtigungen fehlen.

```
nikki@Reihe4-PC2:/home$ ll /usr/local/bin/backItUp.sh
-r--r--rw- 1 root root 638 M r 25  2022 /usr/local/bin/backItUp.sh
```

Damit jeder Nutzer und so auch jedes script die Datei */usr/local/bin/backItUp.sh* ausführen kann, können zusätzlich die Berechtigung *u+x* gesetzt werden.

```
nikki@Reihe4-PC2:/home$ sudo chmod +x /usr/local/bin/backItUp.sh
nikki@Reihe4-PC2:/home$ ll /usr/local/bin/backItUp.sh
-r-xr-xrwx 1 root root 638 M r 25  2022 /usr/local/bin/backItUp.sh
```

In dem Verzeichnis '/home/nikki/bad_passwords_2017/' werden alle Dateien und Ordnern 'u+rx' gesetzt. Der Hinweis, der in allen Dateien hinterlassen wurde, lautet : 'Dieses Passwort sollten Sie auch nicht nutzen.'.

```
nikki@Reihe4-PC2:/home$ chmod -R u+rx /home/nikki/bad_passwords_2017/
insgesamt 40
dr-x------ 2 nikki nikki 4096 Okt 24 12:07 top00-10
dr-x------ 2 nikki nikki 4096 Okt 24 12:07 top10-20
dr-x------ 2 nikki nikki 4096 Okt 24 12:07 top20-30
dr-x------ 2 nikki nikki 4096 Okt 24 12:07 top30-40
dr-x------ 2 nikki nikki 4096 Okt 24 12:07 top40-50
dr-x------ 2 nikki nikki 4096 Okt 24 12:07 top50-60
dr-x------ 2 nikki nikki 4096 Okt 24 12:07 top60-70
dr-x------ 2 nikki nikki 4096 Okt 24 12:07 top70-80
dr-x------ 2 nikki nikki 4096 Okt 24 12:07 top80-90
dr-x------ 2 nikki nikki 4096 Okt 24 12:07 top90-100
nikki@Reihe4-PC2:/home$ cat /home/nikki/bad_passwords_2017/top00-10/
12345      123456     1234567    12345678   123456789  football   letmein    password
    qwerty
nikki@Reihe4-PC2:/home$ cat /home/nikki/bad_passwords_2017/top00-10/12345
Dieses Passwort sollten Sie auch nicht nutzen.
nikki@Reihe4-PC2:/home$ cat /home/nikki/bad_passwords_2017/top00-10/123456
Dieses Passwort sollten Sie auch nicht nutzen.
nikki@Reihe4-PC2:/home$ cat /home/nikki/bad_passwords_2017/top00-10/1234567
Dieses Passwort sollten Sie auch nicht nutzen.
nikki@Reihe4-PC2:/home$ cat /home/nikki/bad_passwords_2017/top00-10/12345678
Dieses Passwort sollten Sie auch nicht nutzen.
nikki@Reihe4-PC2:/home$ cat /home/nikki/bad_passwords_2017/top00-10/123456789
Dieses Passwort sollten Sie auch nicht nutzen.
```

3.4 Gruppenprivilegien

Die Gruppenprivilegien können auch mit den Befehlen *chown* und *chmod* angepasst werden.

Die Nutzer 'alice' und 'bob' werden der Nutzergruppe 'ledx' hinzugefügt.

```
nikki@Reihe4-PC2:~$ sudo usermod -aG ledx alice
nikki@Reihe4-PC2:~$ sudo usermod -aG ledx bob
```

Mit dem Befehl *chwon* wird der Besitzer des Projekt-Ordners '/srv/ledx-projekt/' auf nikki:ledx gesetzt.

```
nikki@Reihe4-PC2:~$ sudo chown nikki:ledx /srv/ledx-projekt
```

Mit dem Befehl *echo "ja ,kann er" >/srv/ledx-projekt/testrechnung.txt* wird eine Test-Datei erstellt und sie wird nach Besitzer und Berechtigungen untersucht. Die Berechtigungen auf der Datei sind *-rw-rw-r–*. Dies sagt aus, dass der Inhaber der Datei, in diesem Fall 'nikki', und die Nutzergruppe 'nikki' Lese- und Schreibrechte hat. Alle anderen Nutzer können die Datei nur Lesen.

```
nikki@Reihe4-PC2:~$ echo "ja, kann er" > /srv/ledx-projekt/testrechnung.txt
nikki@Reihe4-PC2:~$ ll /srv/ledx-projekt/
insgesamt 4
-rw-rw-r-- 1 nikki nikki 12 Okt 25 10:16 testrechnung.txt
```

Mit dem Befehl *rm /srv/ledx-projekt/testrechnung.txt* wird die erzeugte Datei gelöscht. Es wird das GUID-Bit, mit dem Befehl *sudo chmod g+s /srv/ledx-projekt*, auf den Ordner gesetz. Der Besitzer ist weiterhin 'nikki', aber die Gruppenzugehörigkeit des Ordners liegt bei 'ledx'. Wenn nun eine neue Datei mit *echo "ja ,kann er" >/srv/ledx-projekt/testrechnung.txt* erstellt wird, unterscheidet sich die Gruppenzugehörigkeit der Datei. Dies geschiet durch das GUID-Bit. Es bedeutet, dass alle Mitglieder der Gruppe *ledx* Zugriff auf die Dateien in */srv/ledx-projekt* haben.

```
nikki@Reihe4-PC1:~$ rm /srv/ledx-projekt/testrechnung.txt
nikki@Reihe4-PC1:~$ sudo chmod g+s /srv/ledx-projekt/
nikki@Reihe4-PC1:~$ ll /srv/
insgesamt 4
drwxrwsrwx 2 nikki ledx 4096 Okt 25 10:31 ledx-projekt
nikki@Reihe4-PC1:~$ echo "ja, kann er" > /srv/ledx-projekt/testrechnung.txt
nikki@Reihe4-PC1:~$ ll /srv/ledx-projekt/
insgesamt 4
-rw-rw-r-- 1 nikki ledx 12 Okt 25 10:35 testrechnung.txt
```

Die Berechtigung auf den Ordner wird so gesetzt, dass weder der Nutzer 'nikki' noch der Nutzer 'jonas', aber Mitglieder der Gruppe 'ledx' auf den Ordner voll zugreifen können. Mit dem Befehl *chmod 2070 /srv/ledx-projekt* wird dem Besitzer des Verzeichnis' der Zugriff entzogen. Genau so allen anderen Nutzer. Nur Mitglieder der Gruppe, welcher das Verzeichnis gehört, haben vollen Zugriff.

```
nikki@Reihe4-PC2:/home$ chmod 2070 /srv/ledx-projekt
nikki@Reihe4-PC2:/home$ ll /srv/
insgesamt 4
d---rws--- 2 nikki ledx 4096 Okt 25 10:17 ledx-projekt
```

Es kann überprüft werden, ob die Berechtigungen erfolgreich gestzt sind, indem jeder Nutzer versucht in das Verzeichnis zu wechseln. Das ist nur erlaubt, wenn die Berechtinung x gesetzt ist. Durch den Befehl *su -l* kann ein Befehl als ein bestimmter Nutzer ausgeführt werden, ohne eine Konsole im Kontext des Nutzers zu haben. In der Ausgabe kann gesehen werden, dass die Nutzer 'alice' und 'bob' Zugriff auf das Verzeichnis haben müssen, da keine Fehlermeldung ausgegeben wurde. Anders bei den Nutzern 'nikki' und 'jonas'. Hier wurde bei dem Versuch den Inhalt des Verzeichnis' auszugeben ein Fehler ausgegeben, der besagt, dass nicht die nötigen Berechtigungen bestehen.

```
nikki@Reihe4-PC2:/home$ sudo su -l nikki -c "ls -l /srv/ledx-projekt"
-bash: Zeile 1: cd: /srv/ledx-projekt: Keine Berechtigung
nikki@Reihe4-PC2:/home$ sudo su -l alice -c "ls -l /srv/ledx-projekt"
nikki@Reihe4-PC2:/home$ sudo su -l bob -c "ls -l /srv/ledx-projekt"
nikki@Reihe4-PC2:/home$ sudo su -l jonas -c "ls -l /srv/ledx-projekt"
-b4sh: Zeile 1: cd: /srv/ledx-projekt: Keine Berechtigung
```

Ausgabe gekürtzt um Verzeichnisinhalt

Der Benutzer Jonas hat nicht die nötigen Berechtigungen um auf den Ordner zuzugreifen. Da der Nutzer nicht auf den Ordner zugreifen kann, kann dieser auch keine Dateien im Ordner Lesen, da sonst die Zugriffsrechte keine Bedeutung hätten.

```
nikki@Reihe4-PC2:/home$ sudo su -l jonas -c "cat /srv/ledx-projekt/testrechnung.txt"
-b4sh: Zeile 1: cd: /srv/ledx-projekt/testrechnung.txt: Keine Berechtigung
```

3.5 Fallbeispiel cron

In diesem Fallbeispiel wird betrachtet, wie falsche Dateiberechtigungen von einem Angreifer ausgenutzt werden können.

In */etc/crontab* wird "Hier koennte Schadcode stehen" reingeschrieben. In */etc/crontab* kann geschrieben werden, da die Datei dem Nutzer 'nikki' gehört und für den Besitzer die Berechtigungen *rw-* gestzt sind.

```
nikki@Reihe4-PC2:/home$ echo "#Hier koennte Schadcode stehen" >> /etc/crontab
nikki@Reihe4-PC2:/home$ ll /etc/crontab
-rw-rw-r-- 1 nikki nikki 553 Okt 25 10:34 /etc/crontab
```

Der Befehl *chown* wird verwendet, um dies zu korrigieren, und anschließend wird überprüft, ob die Korrektur richtig umgesetzt wurde.

```
nikki@Reihe4-PC2:/home$ sudo chown root:root /etc/crontab
[sudo] Passwort für nikki:
nikki@Reihe4-PC2:/home$ echo "#Hier koennte Schadcode stehen" >> /etc/crontab
-bash: /etc/crontab: Keine Berechtigung
```

Mit *sudo nano /etc/crontab* wird 'Hier koennte weiterer Schadcode stehen' reingeschrieben. Normale Benutzer sollten keine sudo-Berechtigungen haben, damit so ein 'Schadcode' nicht reingeschrieben werden kann.

```
nikki@Reihe4-PC2:/home$ sudo nano /etc/crontab
nikki@Reihe4-PC2:/home$ sudo cat /etc/crontab
# /etc/crontab: system-wide crontab

SHELL=/bin/sh
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

# Example of job definition:
# .---------------- minute (0 - 59)
# |  .------------- hour (0 - 23)
# |  |  .---------- day of month (1 - 31)
# |  |  |  .------- month (1 - 12) OR jan,feb,mar,apr ...
# |  |  |  |  .---- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fri,sat
# |  |  |  |  |
# *  *  *  *  * user-name command to be executed

0 23    * * *   root    /usr/local/bin/backItUp.sh

#Hier koennte Schadcode stehen
#Hier koennte weiterer Schadcode stehen
```

Das in */etc/crontab* angegebene Skript */usr/local/bin/backItUp.sh* wird jeden Tag um 23 Uhr als Nutzer 'root' ausgeführt.

Auch wenn die Datei */etc/crontab* nicht direkt verändert werden kann, ist das Skript */usr/local/bin/backItUp.sh* nicht richtig geschützt. Jeder Nutzer kann das Skript bearbeiten und so auch Schadcode einfügen. Da das Skript weiterhin immer um 23 Uhr ausgeführt wird, wird so auch der Schadcode immer um 23 Uhr ausgeführt.

```
nikki@Reihe4-PC2:/home$ ll /usr/local/bin/backItUp.sh
-r-xr-xrwx 1 root root 638 M r 25  2022 /usr/local/bin/backItUp.sh
nikki@Reihe4-PC2:/home$ sudo chmod 550 /usr/local/bin/backItUp.sh
nikki@Reihe4-PC2:/home$ ll /usr/local/bin/backItUp.sh
-r-xr-x--- 1 root root 638 M r 25  2022 /usr/local/bin/backItUp.sh
```

Nachdem die Berechtigungen auf *-r-xr-x—* geändert werden, kann alleine der Nutzer 'root' und Mitglieder der Gruppe 'root' das Skript anschauen und ausführen. Auch kann nun nicht mehr in das Skript geschrieben werden.

3.6 Fallbeispiel shadow

In diesem Fallbeispiel wird die Bedeutung des Schutzes der Textdatei '/etc/shadow' betrachtet, weshalb besondere Aufmerksamkeit auf die Berechtigungen dieser Datei gelegt werden muss.

```
nikki@Reihe4-PC2:/home$ ll /etc/shadow
-rw-r--rw- 1 root shadow 1773 Okt 24 12:07 /etc/shadow
```

Die Berechtigung von der Datei ist '-rw-r–rw-', also darf die Datei von allen Nutzern gelesen werden.

```
nikki@Reihe4-PC2:/home$ cat /etc/shadow
root:$6$E9B.
    uuqHXtpDTpwx$pRl44lSLZAKKoVIhoPVbESwoqrNkMo9sFNHFCqCJoePcjWkTRvd66BdGVi9us4Zogrj5XM3
[...]
student:$y$j9T$UyRlqBji3D8bIZlGZcYyA1$uIACKneK8Zrpo1yx9tPw4VJ9xfHe0xoxA1dp.SwKBi5
    :19423:0:99999:7:::
sshd:*:19423:0:99999:7:::
nikki:$6$F.Wiz.U8VAhWbfs0$dOVdICNmoPL7LI21eg//dfFI/
    RWu1xA9rQuH9gBbxgHQxsBXeerKTtuyQFWsepAnKB37hEcvT5JjPuL2kFF.41:19654:0:99999:7:::
alice:*:19654:0:99999:7:::
bob:*:19654:0:99999:7:::
jonas:*:19654:0:99999:7:::
nikki@Reihe4-PC2:/home$ ll /etc/shadow
-rw-r--rw- 1 root shadow 1773 Okt 24 12:07 /etc/shadow
```

Ausgabe gekürtzt um vom System und Programmen erschaffene Nutzer ohne Passwort
In der Datei */etc/shadow* sind alle Nutzer mit dem dazugehörigen Passwort gespeichert. Das Passwort liegt in einer gehasheden Version vor, damit es nicht einsehbar ist. Hashfunktionen haben eine Eingabe beliebiger länge und haben eine feste Ausgabelänge. Da Hashfunktionen auch Einwegfunktionen sind, kann das Passwort nicht aus dem Hashwert errechnet werden und ist geschützt. Meist wird eine zufällige Zeichenkette, der Salt, an das Passwort angehängt, um den Hashwert zu verändern. Dies ist hilfreich, da eine einmal berechnete Liste Liste von Passwort-Hash Paaren so nutzlos wird. Das gehashte Passwort von 'nikki' lautet 'dOVdICNmoPL7LI21eg//dfFI/RWu1xA9rQuH9gBbxgHQxsBXeerKTtuyQFWsepAnKB37hEcvT5J jPuL2kFF.41'. Zudem lautet der verwendete Salt 'F.Wiz.U8VAhWbfs0'

```
nikki:$6$F.Wiz.U8VAhWbfs0$dOVdICNmoPL7LI21eg//dfFI/RWu1xA9rQuH9gBbxgHQxsBXeerK
TtuyQFWsepAnKB37hEcvT5JjPuL2kFF.41:19654:0:99999:7:::
```

'o+w' bzw. Schreibberechtigung für 'Andere' bei der Datei '/etc/shadow' ist ein ernstes Sicherheitsproblem, da der Angreifer zum Beispiel Passwort-Hashes verändern kann, um sich selbst Zugriff auf Benutzerkonten zu verschaffen. Er kann einen Hash vorberechnen und in die Datei einfügen, um das alte Passwort zu überschreiben und gegen ein eigenes, bekanntes Passwort zu tauschen.

```
nikki@Reihe4-PC2:/home$ ll /etc/shadow
-rw-r--rw- 1 root shadow 1773 Okt 24 12:07 /etc/shadow
```

Die fehlerhafte Konfiguration wird zu '-rw-r——' verändert und somit korigiert. Die Passwort-Hashes können von dem Nutzer 'nikki' nicht mehr gelesen werden.

```
nikki@Reihe4-PC2:/home$ sudo chmod 640 /etc/shadow
nikki@Reihe4-PC2:/home$ ll /etc/shadow
-rw-r----- 1 root shadow 1773 Okt 24 12:07 /etc/shadow
nikki@Reihe4-PC2:/home$ cat /etc/shadow
cat: /etc/shadow: Keine Berechtigung
```

4 Zusammenfassung und Fazit

Durch diesen Versuch ist klar geworden, wie wichtig Berechtigungen und Privilegien sind. Nutzer brauchen auf bestimmte Verzeichnisse Zugriff, alleine um richtig arbeiten zu können. Wenn der Zugriff auf das eigene Home-Verzeichnis nicht gegeben ist, können keine Dateien angelegt, gelöscht oder bearbeitet werden, obwohl es möglich seien sollte. Zudem können die falschen Berechtigungen fatale Folgen haben, wie Schadcode in einer Datei, welche mit root Privilegien periodisch ausgeführt wird. Auch die generelle Sicherheit spielt eine Rolle. Wenn ein Angreifer durch falsch gesetzte Privilegien an eine Datei wie */etc/shadow* kommt, kann er möglicherweise die Passwörter der Nutzer in Erfahrung bringen. Da Menschen ein gewähltes Passwort oft mehrmals verwendet, kann der Angreiffer sich möglicherweise auf verschiedenen Internetseiten mit diesem anmelden und so Probleme verursachen.

In diesem Versuch ist deutlich geworden, wie Privilegien genutzt werden können um Probleme zu vermeiden, wie Dateien nur ausgewählten Nutzer zugänglich gemacht werden können und wie ein strukturiertes Arbeitsumfeld durch Nutzergruppen geschaffen werden kann.

Die Durchführung dieses Versuches war interessant und lehrreich. Wir haben viele wichtige Techniken kennen gelernt, die in Alltagssituationen am Computer hilfreich seien werden. Trotz einiger Schwierigkeiten und Hindernisse ist es uns erfolgreich gelungen, die Aufgaben abzuschließen. Durch die gute Erklärung in der Aufgabenstellung haben wir die Aufgaben schnell verstanden. Leider gab es auch zwischenzeitig Probleme, bei denen uns nicht geholfen werden konnte, bis sie selbst gelöst bekommen haben. Zusammengefasst war der Versuch sehr nützlich und gut durchdacht.

5 LLM

In diesem Versuch wurden keine Large-Language Models(LLM) verwendet. Die Texte sind alleine die Arbeit der Mitglieder der Gruppe 43 wie zubeginn angegeben.

Literatur

[1] Ruhr-Universität Bochum Lehrstuhl für Netz-und Datensicherheit. Bachelor Grundlagenpraktikum ITS: Veranstaltungsseite der Fakultät für Elektrotechnik und Informationstechnik. URL `https://informatik.rub.de/studium/lehrveranstaltungen/`.

[2] Ruhr-Universität Bochum Lehrstuhl für Netz-und Datensicherheit. Aufgabenstellung zum Praktikumsversuch. Bochum, 2023.